

TIME TAXI KHAOLAK

Clone Project — Technical Documentation

Phase 0 + Phase 1 Complete

เอกสาร	รายละเอียด
ชื่อโปรเจกต์	taxi-booking — Clone of timetaxikhaolak.com
เทคโนโลยีหลัก	Next.js 16 + TypeScript + Prisma 7 + PostgreSQL
สถานะ	Phase 1 Complete — พร้อมเริ่ม Phase 2
วันที่จัดทำ	9 พฤษภาคม 2569
Deployment	Option B — VPS Docker Compose (DigitalOcean Singapore)

1. ภาพรวมโปรเจกต์

โปรเจกต์นี้เป็นการ clone ระบบเว็บไซต์จองแท็กซี่ส่วนตัว Time Taxi Khaolak (timetaxikhaolak.com) ซึ่งให้บริการรับส่งส่วนตัวระหว่างสนามบินภูเก็ตและเขาหลัก โดยระบบมีทั้งหมด 8 หน้า พร้อมระบบจองออนไลน์ และ Admin Panel สำหรับจัดการการจอง

1.1 หน้าเว็บที่ต้อง Clone

URL	หน้าที่
/	Home — Landing page, hero, vehicle types, reviews, FAQ
/booking	ฟอร์มจองรถออนไลน์ 12 fields
/price-list	ตารางราคา 6 โซนพื้นที่ (SSG)
/about	เกี่ยวกับบริษัท, บริการ, ทีมงาน
/qa	คำถามที่พบบ่อย accordion
/reviews	รีวิวจากลูกค้า
/travel	แนะนำสถานที่ท่องเที่ยว
/admin	Admin Panel — จัดการการจอง, ราคา, รีวิว

1.2 จุดเด่นของระบบ

- Pay After Trip — No Deposit Required (ไม่ต้องจ่ายล่วงหน้า)
- รองรับ 2 ภาษา — ไทย / English
- ติดต่อผ่าน LINE, WhatsApp, WeChat, Email
- Vehicle Types: Sedan (1-2 pax), SUV (1-4 pax), Van (1-9 pax)
- Admin dashboard จัดการ booking status
- Email notification เมื่อมีการจองใหม่

2. Tech Stack ที่เลือกใช้

เลือก Deployment Option B: VPS Docker Compose บน DigitalOcean Singapore เนื่องจากถูกกว่า Vercel 40%, latency ดีกว่าสำหรับผู้ใช้ในไทย, ไม่มี vendor lock-in, และ PostgreSQL รันตลอดไม่มี cold start

2.1 Stack หลัก

Layer	Technology
Frontend Framework	Next.js 16.2.6 (App Router + Turbopack)
Language	TypeScript 5.9.3
Styling	Tailwind CSS v4.2.4
UI Components	shadcn/ui (Radix + Nova preset)
ORM	Prisma 7.8.0
Database	PostgreSQL 16 (Docker)
Authentication	NextAuth.js v5 (beta)
Form Validation	react-hook-form 7 + zod 4
Email	Nodemailer 8
Package Manager	pnpm 11.0.9
Runtime	Node.js 22.22.2
Deployment	Docker Compose + Nginx + Let's Encrypt

2.2 เปรียบเทียบ Deployment Options

ปัจจัย	Option B: VPS DigitalOcean (ที่เลือก)
ค่าใช้จ่าย/เดือน	\$12/เดือน คงที่
ค่าใช้จ่าย 3 ปี	\$432 (ประหยัดกว่า Vercel \$288)
Latency จากไทย	~20-30ms (Singapore)
Cold Start	ไม่มี — PostgreSQL รันตลอด
Vendor Lock-in	ไม่มี — ย้ายได้ทุกเวลา
CI/CD	GitHub Actions — push แล้ว deploy อัตโนมัติ

3. Phase 0 — Setup & เรียน Basics (3-4 วัน)

Phase 0 เน้นการเตรียมเครื่องและเรียนรู้ concept ใหม่ก่อนเริ่ม code จริง

3.1 สิ่งที่ต้องตั้ง

- Node.js 22.22.2 LTS (ผ่าน nvm-windows)
- pnpm 11.0.9 — package manager เร็วกว่า npm ~3x
- VS Code Extensions: Tailwind IntelliSense, Prisma, ESLint, Prettier, Error Lens, Thunder Client, GitLens

3.2 TypeScript Concepts ที่ต้องรู้

Concept	ตัวอย่าง
Basic Types	<code>const name: string = 'Mai'</code>
Interface	<code>interface Booking { name: string; phone: string }</code>
Union Type	<code>type VehicleType = 'sedan' 'suv' 'van'</code>
Optional	<code>email?: string</code>
Function Types	<code>function calc(zone: string): number { }</code>
Generics	<code>Promise<Booking[]></code>

3.3 React Concepts (เทียบกับ Angular)

Angular	React
<code>@Component + class</code>	Function Component
<code>@Input() prop</code>	<code>props: { name: string }</code>
<code>this.value (state)</code>	<code>useState()</code>
<code>ngOnInit()</code>	<code>useEffect(() => {}, [])</code>
<code>*ngIf="condition"</code>	<code>{condition && <div>}</code>
<code>*ngFor="let x of list"</code>	<code>{list.map(x => <div key={x.id}>)}</code>
<code>[(ngModel)]</code>	<code>value={x} onChange={setX}</code>

3.4 Git Branch Strategy

- main — production branch (GitHub Actions deploy อัตโนมัติ)
- dev — development integration branch
- feature/xxx — แต่ละ feature แยก branch

Commit convention: feat: / fix: / chore: / docs:

4. Phase 1 — Project Foundation (2-3 วัน)

Phase 1 สร้าง foundation ของโปรเจกต์ — Next.js project, packages, folder structure, Docker local DB, Tailwind theme, และ Prisma setup

4.1 การสร้าง Next.js Project

```
คำสั่ง: pnpm create next-app@latest . --typescript --tailwind --eslint --app --src-dir --import-alias "@/*" --use-pnpm
```

หมายเหตุพิเศษสำหรับ pnpm 11: ต้องสร้าง pnpm-workspace.yaml ก่อนเพื่อ allow build scripts:

onlyBuiltDependencies:

- sharp
- unrs-resolver

4.2 Packages ที่ติดตั้ง

Package	Version	หมวด	หน้าที่
@prisma/client	7.8.0	Database	Prisma ORM client
prisma	7.8.0	Database (dev)	CLI + Studio
@prisma/adapters-pg	7.8.0	Database	PostgreSQL adapter (Prisma 7)
pg	8.20.0	Database	PostgreSQL driver
dotenv	17.4.2	Config	Load env vars สำหรับ Prisma CLI
react-hook-form	7.75.0	Forms	Form state management
@hookform/resolvers	5.2.2	Forms	เชื่อม RHF กับ zod
zod	4.4.3	Validation	Schema validation + TypeScript types
next-auth	5.0.0-beta.31	Auth	Authentication สำหรับ admin
nodemailer	8.0.7	Email	ส่ง email notification
radix-ui	1.4.3	UI	Headless UI components
shadcn	4.7.0	UI	shadcn/ui component system
tw-animate-css	1.4.0	UI	Tailwind animations
clsx	2.1.1	Utilities	Conditional class names
tailwind-merge	3.5.0	Utilities	Merge Tailwind classes
lucide-react	1.14.0	Icons	Icon library

4.3 Folder Structure

โครงสร้าง folder ที่สร้างใน Phase 1:

Path	หน้าที่
<code>src/app/(public)/</code>	หน้าสาธารณะทั้งหมด (shared Navbar+Footer layout)
<code>src/app/admin/</code>	Admin panel — ต้อง login ก่อนเข้า
<code>src/app/api/booking/</code>	Route Handler: POST /api/booking
<code>src/app/api/auth/[...nextauth]/</code>	NextAuth.js endpoints
<code>src/components/ui/</code>	shadcn/ui components (auto-generated)
<code>src/components/layout/</code>	Navbar.tsx, Footer.tsx
<code>src/components/home/</code>	HeroSection, VehicleCards, FaqAccordion
<code>src/components/booking/</code>	BookingForm, VehicleSelector
<code>src/lib/db.ts</code>	Prisma client singleton (Prisma 7 adapter pattern)
<code>src/lib/utils.ts</code>	cn() helper สำหรับ merge Tailwind classes
<code>src/types/index.ts</code>	Shared TypeScript types ทั้งโปรเจกต์
<code>src/validations/booking.ts</code>	Zod schema สำหรับ booking form
<code>prisma/schema.prisma</code>	Database schema (Prisma 7)
<code>prisma.config.ts</code>	Prisma 7 config — DATABASE_URL + migrations path
<code>docker-compose.dev.yml</code>	PostgreSQL + pgAdmin สำหรับ local dev
<code>.env.local</code>	Environment variables (ไม่ commit)
<code>.env.example</code>	Template env vars (commit ได้)

4.4 Tailwind CSS v4 Custom Theme

ตั้งค่า custom colors ใน `src/app/globals.css` ด้วย `@theme`:

Color Variable	Hex Value / ใช้สำหรับ
<code>--color-navy-900</code>	<code>#0A1628</code> — Background หลัก
<code>--color-navy-800</code>	<code>#0F2040</code> — Card background
<code>--color-navy-700</code>	<code>#1A3A5C</code> — Border color
<code>--color-gold-500</code>	<code>#D4AF37</code> — Accent / CTA หลัก
<code>--color-gold-400</code>	<code>#F4C430</code> — Hover state
<code>--color-gold-600</code>	<code>#B8962E</code> — Active state

4.5 Prisma 7 — Breaking Changes ที่แก้ไข

ปัญหา: Prisma 7 ย้าย `url = env("DATABASE_URL")` ออกจาก `schema.prisma` ไปอยู่ใน `prisma.config.ts` แทน

การแก้ไข — สร้าง `prisma.config.ts` ที่ root:

```
import dotenv from 'dotenv'
dotenv.config({ path: '.env.local' })
import { defineConfig } from 'prisma/config'
export default defineConfig({
  schema: 'prisma/schema.prisma',
  datasource: { url: process.env['DATABASE_URL'] },
  migrations: { path: 'prisma/migrations' },
})
```

schema.prisma (url ออกแล้ว):

```
datasource db {
  provider = "postgresql"
  // url ย้ายไปอยู่ใน prisma.config.ts แล้ว
}
```

db.ts — ใช้ adapter pattern แบบใหม่ของ Prisma 7:

```
import { PrismaPg } from '@prisma/adapter-pg'
const adapter = new PrismaPg({ connectionString: process.env.DATABASE_URL! })
export const db = new PrismaClient({ adapter })
```

4.6 Docker Compose (Local Development)

รัน PostgreSQL + pgAdmin ด้วย docker-compose.dev.yml:

Container	รายละเอียด
taxi_db_dev	postgres:16-alpine — port 5432
taxi_pgadmin	pgAdmin 4 — port 8080 (localhost:8080)

- pgAdmin login: admin@taxi.dev / admin123
- Database: taxi_booking, User: taxi_user

4.7 สรุป Checklist Phase 1

รายการ	สถานะ
Next.js 16 + TypeScript project สร้างสำเร็จ	✓ เสร็จ
pnpm-workspace.yaml แก้ปัญหา pnpm 11 build scripts	✓ เสร็จ
ติดตั้ง packages ครบทุก group	✓ เสร็จ
shadcn/ui init (Radix + Nova preset)	✓ เสร็จ
Folder structure ครบตาม plan	✓ เสร็จ
Docker — taxi_db_dev + taxi_pgadmin running	✓ เสร็จ
Tailwind v4 custom theme (navy + gold)	✓ เสร็จ

Base files: db.ts, utils.ts, types, validations	✓ เสร็จ
prisma.config.ts แก้ปัญหา Prisma 7 breaking change	✓ เสร็จ
npm prisma generate สำเร็จ	✓ เสร็จ
npm prisma db push — database in sync	✓ เสร็จ
localhost:3000 แสดง dark navy + gold theme	✓ เสร็จ

5. Phase 2 — Database Schema (ถัดไป)

Phase 2 จะเขียน Prisma schema ครบทั้ง 8 tables และ seed ข้อมูลตัวอย่าง

Table	หน้าที่
bookings	เก็บการจองทั้งหมด — 17 fields
price_zones	โซนพื้นที่ 6 โซน (Phuket, Surat Thani, Krabi ฯลฯ)
price_routes	ราคาแต่ละเส้นทาง (Standard / SUV / Van)
reviews	รีวิวลูกค้า พร้อม source และ rating
faqs	Q&A รองรับ TH/EN
attractions	สถานที่แนะนำ (Travel page)
admin_users	Admin accounts พร้อม bcrypt
settings	Key-value config (WhatsApp number, contact info)

5.1 Road Map ทั้งหมด

Phase	เนื้อหา / ระยะเวลา
Phase 0 	Setup + React/TypeScript basics (3-4 วัน)
Phase 1 	Next.js project + Docker + Tailwind + Prisma (2-3 วัน)
Phase 2	Database schema 8 tables + seed data (2 วัน)
Phase 3	Public pages — Home, Price List, About, Q&A (5-7 วัน)
Phase 4	Booking form + API route + Email notification (3-4 วัน)
Phase 5	Admin panel — Dashboard + CRUD (4-5 วัน)
Phase 6	i18n (TH/EN) + SEO + Mobile optimization (2-3 วัน)
Phase 7	Production deploy — VPS + GitHub Actions + SSL (2-3 วัน)

ประมาณการรวม: 5-6 สัปดาห์ (ทำวันละ 2-4 ชั่วโมง)

6. Environment Variables

ไฟล์ .env.local (ไม่ commit ขึ้น Git):

Variable	ค่าและหน้าที่
DATABASE_URL	postgresql://taxi_user:taxi_pass_dev@localhost:5432/taxi_booking
NEXTAUTH_SECRET	random string — ใช้ sign JWT tokens
NEXTAUTH_URL	http://localhost:3000 (dev) / https://domain.com (prod)
EMAIL_USER	Gmail address สำหรับส่ง notification
EMAIL_PASS	Gmail App Password (ไม่ใช่ password จริง)
EMAIL_TO	อีเมลรับ notification เมื่อมีการจองใหม่
ADMIN_EMAIL	Admin login email
ADMIN_PASSWORD	Admin login password (เปลี่ยนก่อน deploy)

— End of Document —